

Documentacion de integracion: React + Spring Boot + MySQL

Proyecto DistriRepuestos / MotoRepuestos

Objetivo	Explicar que se hizo, como se hizo y por que se eligio este metodo de integracion.
Frontend	React + Vite, conservando la interfaz y componentes existentes.
Backend	Spring Boot 2.7.x con API REST y JPA.
Base de datos	MySQL con tablas generadas por Hibernate y carga inicial de productos.

1. Que se hizo

El proyecto original era una aplicacion React que simulaba varias funciones en el navegador. Los productos venian de un archivo local, el carrito se guardaba en localStorage, el login era simulado, los pedidos vivian en estado de React y el formulario de contacto tambien se guardaba localmente.

La modificacion principal fue agregar un backend Spring Boot conectado a MySQL y cambiar el frontend para usar una API REST, sin perder el flujo visual que ya tenia la aplicacion.

Funcion	Antes	Ahora
Catalogo	src/data.js en React	GET /api/products desde Spring Boot y MySQL
Login/registro	Simulado con alert y estado local	POST /api/auth/login y /api/auth/register
Pedidos	Estado temporal en React	POST /api/orders y consulta por correo
Contacto	localStorage	POST /api/contact persistido en MySQL
Carrito	localStorage	Se conserva local porque es estado temporal del navegador

2. Por que no se convirtio React completamente a Spring Boot

Una duda normal es pensar que pasar a Spring Boot significa borrar React y hacer todas las pantallas con Java. No elegi ese camino porque no era el mas conveniente para tu proyecto.

- React ya tenia la interfaz, componentes, estilos, modales, filtros y experiencia de usuario funcionando.
- Spring Boot es mucho mejor como backend: API, reglas de negocio, conexion a base de datos y persistencia.
- Separar frontend y backend permite mantener la parte visual sin rehacer todo desde cero.
- Es la arquitectura mas comun en proyectos modernos: React consume servicios REST hechos en Spring Boot.
- Rehacer todas las vistas con Thymeleaf u otro motor de plantillas habria tomado mas tiempo y habria duplicado trabajo ya hecho.

3. Metodo elegido

Use una arquitectura de dos capas principales:

1. Frontend React/Vite: se encarga de mostrar productos, carrito, checkout, login, cuenta y contacto.
2. Backend Spring Boot: expone endpoints REST, valida datos, guarda en MySQL y entrega informacion al frontend.
3. MySQL: almacena productos, usuarios, pedidos, items de pedido y mensajes de contacto.

4. Backend Spring Boot creado

Se creo la carpeta motorepuestos-backend con una estructura tipica de Spring Boot:

```
motorepuestos-backend/  
  pom.xml  
  src/main/resources/application.properties  
  src/main/java/com/distrirepuestos/  
    MotorepuestosBackendApplication.java  
  config/  
  controller/  
  dto/  
  model/  
  repository/
```

4.1 Dependencias

El archivo pom.xml usa Spring Boot 2.7.18 porque la maquina tenia Java 8 instalado. Spring Boot 3 exige Java 17 o superior.

- spring-boot-starter-web: crear controladores REST.
- spring-boot-starter-data-jpa: conectar entidades Java con tablas MySQL.
- spring-boot-starter-validation: validar datos de entrada.
- mysql-connector-java: driver para conectar con MySQL.

4.2 Entidades principales

Entidad	Para que sirve
Product	Representa cada repuesto del catalogo.
Customer	Representa los usuarios registrados.
SaleOrder	Representa un pedido completo.
SaleOrderItem	Representa los productos dentro de un pedido.
ContactMessage	Representa los mensajes enviados desde el formulario de contacto.

4.3 Repositorios

Los repositorios extienden JpaRepository. Esto evita escribir SQL manual para operaciones basicas como guardar, listar o buscar por id. Spring Data JPA genera esas consultas automaticamente.

4.4 Controladores REST

Endpoint	Funcion
GET /api/products	Lista productos desde MySQL.
POST /api/auth/register	Registra un cliente.
POST /api/auth/login	Valida correo y contrasena.
POST /api/orders	Crea un pedido con sus productos.
GET /api/orders?email=...	Consulta pedidos de un cliente.
POST /api/contact	Guarda mensajes de contacto.

4.5 Carga inicial de productos

Agregue DataSeeder para que, cuando la tabla de productos este vacia, Spring Boot cargue los productos iniciales. Esto permite que el catalogo exista en MySQL sin tener que insertar manualmente los 48 productos uno por uno.

5. Conexion con MySQL

La configuracion quedo en application.properties:

```
server.port=${SERVER_PORT:8080}
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/distrirepuestos?...}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:}
spring.jpa.hibernate.ddl-auto=update
```

Use variables de entorno para que no sea obligatorio escribir usuario y contrasena directamente en el codigo. Esto es mejor practica porque permite cambiar credenciales sin modificar archivos del proyecto.

6. Cambios hechos en React

El frontend se mantuvo, pero ahora llama al backend mediante fetch.

- Se creo src/api.js para centralizar todas las llamadas HTTP.
- App.jsx carga productos desde /api/products y usa datos locales solo como respaldo si la API no esta levantada.
- AuthModal.jsx ahora registra e inicia sesion contra Spring Boot.
- Checkout.jsx crea pedidos reales con POST /api/orders.
- ContactForm.jsx guarda mensajes con POST /api/contact.
- El carrito se mantiene en localStorage porque es informacion temporal del navegador antes de confirmar el pedido.

7. Por que algunas cosas quedaron en React

No todo debe ir a la base de datos. El carrito, por ejemplo, puede quedarse en el navegador hasta que el usuario confirme la compra. Solo cuando se confirma el pedido se envia al backend y se guarda en MySQL.

Elemento	Donde quedo	Razon
Carrito antes de	React/localStorage	Es temporal y mejora la experiencia si se recarga

comprar		la pagina.
Pedido confirmado	Spring Boot/MySQL	Ya es una transaccion importante que debe persistir.
Productos	MySQL	Deben poder administrarse y consultarse desde backend.
Sesion simple	React + API	Se mantuvo simple para respetar el alcance original.

8. Alternativas que no use

Alternativa	Por que no fue la mejor aqui
Rehacer todo con Thymeleaf	Implicaba reconstruir la interfaz que React ya tenia funcionando.
Crear un monolito con React embebido solamente como archivos estaticos	Sirve para despliegue, pero durante desarrollo es mas claro separar Vite y Spring Boot.
Usar Node/Express como backend	El pedido fue Spring Boot y Java es una buena opcion empresarial para MySQL y JPA.
Escribir SQL manual para todo	JPA reduce codigo repetitivo y mantiene entidades organizadas.
Implementar JWT y Spring Security completo	Es mejor para produccion, pero para esta primera integracion habria aumentado mucho la complejidad.

9. Como ejecutar el proyecto

9.1 Crear base de datos

```
CREATE DATABASE distrirepuestos CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

9.2 Backend

```
cd motorepuestos-backend
$env:DB_USER="root"
$env:DB_PASSWORD="tu_password"
mvn spring-boot:run
```

9.3 Frontend

```
cd motorepuestos-app
npm install
npm run dev
```

10. Verificaciones realizadas

- npm run lint paso correctamente despues de corregir advertencias del proyecto.
- npm run build paso correctamente.
- La app React abrio en <http://localhost:5173>.
- No se pudo ejecutar Spring Boot porque la maquina no tenia Maven instalado.
- No se encontro un servicio MySQL activo en Windows durante la revision.

11. Que faltaria para produccion

- Instalar Maven y MySQL para probar el backend completo localmente.
- Agregar Spring Security con BCrypt para no guardar contraseñas en texto plano.
- Agregar JWT o sesiones seguras para autenticacion real.
- Crear panel administrativo para gestionar productos, stock y estados de pedidos.
- Agregar pruebas automatizadas de backend y frontend.

12. Resumen mental para estudiar

La idea clave es esta: React no desaparece; se convierte en el cliente visual. Spring Boot se vuelve el servidor que maneja datos y reglas. MySQL se vuelve la memoria permanente del sistema. El navegador pide informacion a Spring Boot; Spring Boot consulta o guarda en MySQL; React recibe respuestas y actualiza la pantalla.

Base de Datos

Spring Boot crea las tablas automáticamente porque tiene:

`spring.jpa.hibernate.ddl-auto=update`

Las entidades principales son:

-

Product: productos del catálogo.

-

Customer: usuarios/clientes.

-

SaleOrder: pedido principal.

-

SaleOrderItem: productos dentro de un pedido.

-

ContactMessage: mensajes enviados desde contacto.

Cuando el backend inicia por primera vez, DataSeeder.java revisa si hay productos. Si no hay, inserta productos iniciales.

Endpoints Principales

Productos:

GET /api/products

Devuelve todos los productos.

También permite filtrar por categoría:

GET /api/products?category=Motor

Autenticación:

POST /api/auth/register

POST /api/auth/login

Pedidos:

POST /api/orders

GET /api/orders?email=correo@ejemplo.com

Contacto:

POST /api/contact

Flujo Normal De Uso

1.

El usuario abre el frontend en <http://localhost:5173>.

2.

React carga la app desde App.jsx.

3.

La app intenta pedir productos al backend con `api.getProducts()`.

4.

Si el backend responde, muestra productos desde MySQL.

5.

Si el backend falla, usa productos locales de `data.js`.

6.

El usuario filtra, busca, abre productos y agrega al carrito.

7.

El carrito se guarda en `localStorage`.

8.

Si hace checkout, React manda el pedido a `POST /api/orders`.

9.

El backend guarda el pedido y sus productos en MySQL.

10.

Si el usuario inicia sesión, puede consultar sus pedidos por correo.

Cosas A Tener En Cuenta

Hay textos con caracteres dañados, por ejemplo:

ContraseÃ±a

TransmisiÃ³n

ElÃ©ctrico

Eso es un problema de codificación de caracteres. Funciona, pero visualmente se ve mal. Se debería corregir a UTF-8.

También la seguridad es básica:

-

Las contraseñas se guardan en texto plano.

-

No hay JWT.

-

No hay Spring Security.

-

El login solo compara correo y contraseña directamente.

Para un proyecto académico o demo está bien, pero para producción habría que mejorar esa parte.

Resumen Corto

Es una tienda de repuestos de motos con:

-

Catálogo de productos.

-

Filtros y búsqueda.

-

Carrito de compras.

-

Registro e inicio de sesión.

-

Checkout.

-

Historial de pedidos.

-

Formulario de contacto.

-

Backend conectado a MySQL.

Frontend: React/Vite.

Backend: Spring Boot/MySQL.

La conexión entre ambos se hace por endpoints REST bajo /api.